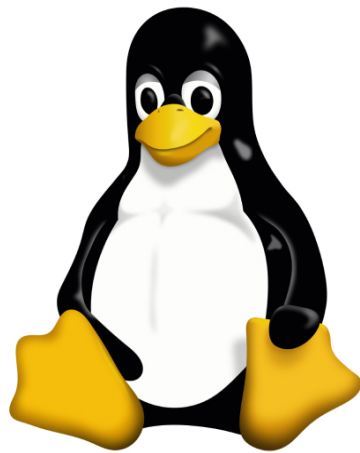


THE LINUX



COMMANDS HANDBOOK

Flavio Copes

Table of Contents

Preface

Introduction to Linux and shells

man

ls

cd

pwd

mkdir

rmdir

mv

cp

open

touch

find

ln

gzip

gunzip

tar

alias

cat

less

tail

wc

grep

sort

uniq

diff

echo

chown

chmod

umask

du

df

basename

dirname

ps

top

kill

killall

jobs

bg

fg

type

which

nohup

xargs

vim

emacs

nano

whoami

who

su

sudo

passwd

ping

traceroute

clear

history

export

crontab

uname

env

printenv

Conclusion

Preface

The Linux Commands Handbook follows the 80/20 rule: learn in 20% of the time the 80% of a topic.

I find this approach gives a well-rounded overview.

This book does not try to cover everything under the sun related to Linux and its commands. It focuses on the small core commands that you will use the 80% or 90% of the time, trying to simplify the usage of the more complex ones.

All those commands work on Linux, macOS, WSL, and anywhere you have a UNIX environment.

I hope the contents of this book will help you achieve what you want: **get comfortable with Linux**.

This book is written by Flavio. I **publish programming tutorials** every day on my website flaviocopes.com.

You can reach me on Twitter [@flaviocopes](https://twitter.com/flaviocopes).

Enjoy!

Introduction to Linux and shells

Linux is an operating system, like macOS or Windows.

It is also the most popular Open Source and free, as in freedom, operating system.

It powers the vast majority of the servers that compose the Internet. It's the base upon which everything is built upon. But not just that. Android is based on (a modified version of) Linux.

The Linux "core" (called *kernel*) was born in 1991 in Finland, and it went a really long way from its humble beginnings. It went on to be the kernel of the GNU Operating System, creating the duo GNU/Linux.

There's one thing about Linux that corporations like Microsoft and Apple, or Google, will never be able to offer: the freedom to do whatever you want with your computer.

They're actually going in the opposite direction, building walled gardens, especially on the mobile side.

Linux is the ultimate freedom.

It is developed by volunteers, some paid by companies that rely on it, some independently, but there's no single commercial company that can dictate what goes into Linux, or the project priorities.

Linux can also be used as your day to day computer. I use macOS because I really enjoy the applications, the design and I also used to be an iOS and Mac apps developer, but before using it I used Linux as my main computer Operating System.

No one can dictate which apps you can run, or "call home" with apps that track you, your position, and more.

Linux is also special because there's not just "one Linux", like it happens on Windows or macOS. Instead, we have **distributions**.

A "distro" is made by a company or organization and packages the Linux core with additional programs and tooling.

For example you have Debian, Red Hat, and Ubuntu, probably the most popular.

Many, many more exist. You can create your own distribution, too. But most likely you'll use a popular one, one that has lots of users and a community of people around it, so you can do what you need to do without losing too much time reinventing the wheel and figuring out answers to common problems.

Some desktop computers and laptops ship with Linux preinstalled. Or you can install it on your Windows-based computer, or on a Mac.

But you don't need to disrupt your existing computer just to get an idea of how Linux works.

I don't have a Linux computer.

If you use a Mac you need to know that under the hood macOS is a UNIX Operating System, and it shares a lot of the same ideas and software that a GNU/Linux system uses, because GNU/Linux is a free alternative to UNIX.

[UNIX](#) is an umbrella term that groups many operating systems used in big corporations and institutions, starting from the 70's

The macOS terminal gives you access to the same exact commands I'll describe in the rest of this handbook.

Microsoft has an official [Windows Subsystem for Linux](#) which you can (and should!) install on Windows. This will give you the ability to run Linux in a very easy way on your PC.

But the vast majority of the time you will run a Linux computer in the cloud via a VPS (Virtual Private Server) like DigitalOcean.

A shell is a command interpreter that exposes to the user an interface to work with the underlying operating system.

It allows you to execute operations using text and commands, and it provides users advanced features like being able to create scripts.

This is important: shells let you perform things in a more optimized way than a GUI (Graphical User Interface) could ever possibly let you do. Command line tools can offer many different configuration options without being too complex to use.

There are many different kind of shells. This post focuses on Unix shells, the ones that you will find commonly on Linux and macOS computers.

Many different kind of shells were created for those systems over time, and a few of them dominate the space: Bash, Csh, Zsh, Fish and many more!

All shells originate from the Bourne Shell, called `sh`. "Bourne" because its creator was Steve Bourne.

Bash means *Bourne-again shell*. `sh` was proprietary and not open source, and Bash was created in 1989 to create a free alternative for the GNU project and the Free Software Foundation. Since projects had to pay to use the Bourne shell, Bash became very popular.

If you use a Mac, try opening your Mac terminal. That by default is running ZSH. (or, pre-Catalina, Bash)

You can set up your system to run any kind of shell, for example I use the Fish shell.

Each single shell has its own unique features and advanced usage, but they all share a common functionality: they can let you execute programs, and they can be programmed.

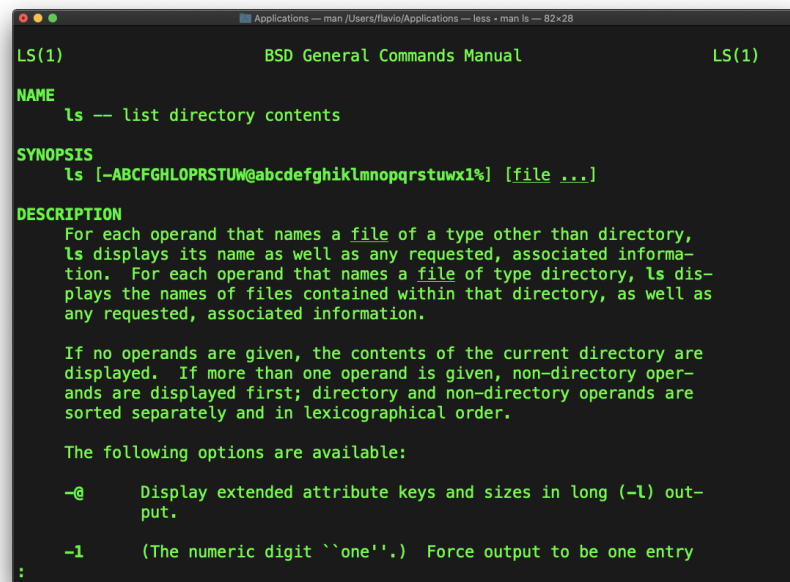
In the rest of this handbook we'll see in detail the most common commands you will use.

man

The first command I want to introduce is a command that will help you understand all the other commands.

Every time I don't know how to use a command, I type

`man <command>` to get the manual:



```
LS(1) BSD General Commands Manual LS(1)

NAME
  ls -- list directory contents

SYNOPSIS
  ls [-ABCFGHLOPRSTUW@abcdefghiklmnopqrstuwx1] [file ...]

DESCRIPTION
  For each operand that names a file of a type other than directory,
  ls displays its name as well as any requested, associated informa-
  tion. For each operand that names a file of type directory, ls dis-
  plays the names of files contained within that directory, as well as
  any requested, associated information.

  If no operands are given, the contents of the current directory are
  displayed. If more than one operand is given, non-directory oper-
  ands are displayed first; directory and non-directory operands are
  sorted separately and in lexicographical order.

  The following options are available:

  -@      Display extended attribute keys and sizes in long (-l) out-
          put.
  -1      (The numeric digit `one'.) Force output to be one entry
  :
```

This is a man (from *manual*) page. Man pages are an essential tool to learn, as a developer. They contain so much information that sometimes it's almost too much.

The above screenshot is just 1 of 14 screens of explanation for the `ls` command.

Most of the times when I'm in need to learn a command quickly I use this site called **tldr pages**: <https://tldr.sh/>. It's a command you can install, then you run it like this: `tldr <command>`, which gives you a very quick overview of a command, with some handy examples of common usage scenarios: